HOSPITAL EMERGENCY RESOURCE MANAGER (HERM)

BCSE303L: Operating Systems | Faculty: Dr. V. Ezhil Arasi | VIT Vellore

REVIEW 2: COMPLETE IMPLEMENTATION & CONFIGURATION DETAILS

Academic Year: 2026–2027 | **School of Computer Science and Engineering**

Project Title:	Hospital Emergency Resource Manager (HERM) Using Operating System Concepts
Team Members:	1. **Gaurav Charpe** (Reg. No: 24BCT0326) 2. **Nitin Sunil Holkar** (Reg. No: 24BCE2819) 3. **Aditya Phogaat** (Reg. No: 24BDS0349)
Programming Language:	**C Language (C11 Standard)** with native POSIX Threads (`pthreads`), Semaphores & Mutexes
Public GitHub Link:	**https://github.com/gauravcharpe/hospital-emergency-resource-manager** (Public, 100% Implemented)
Submission Scope:	Technology Stack, System Architecture, Algorithm Formulations, Configuration Details & Source Code Evidence

1. Technology Stack & Exact System Configuration Details

Subsystem	Technology / Tool	Exact Configuration / Compiler Flags	Role in Project
Core Language	C (C11 / C99)	clang / gcc with -std=c11 -Wall -Wextra -O2	Implements high-performance kernel simulation engine with zero
Multithreading	POSIX Threads (pthreads)	-pthread linker flag, pthread_create(), pthread_join()	Spawns concurrent patient threads and discrete scheduler clock
Synchronization	POSIX Mutex & Semaphores	pthread_mutex_t, pthread_cond_t, custom counting sem	Enforces mutual exclusion on 5 resource pools; prevents double
Build Automation	GNU Make	Makefile targets: all, herm_cli, herm_server, bench, clean	Standardized one-command automated compilation across macO
Networking & API	BSD Sockets (sys/socket.h)	TCP Port 5050, non-blocking HTTP REST daemon	Emits real-time state telemetry JSON to web command center wi
Command Center	HTML5 / CSS3 / ES6	CSS Grid, Flexbox, Canvas SVG WFG visualizer	Aesthetic dark-mode clinical dashboard displaying live queue Ka

2. Algorithmic Formulations Implemented in C

A. Preemptive Priority Scheduling with Dynamic Aging:

• *Immediate Preemption:* If a Level 1 Critical trauma/cardiac patient arrives, running lower-priority tasks are suspended back to READY and contested equipment is reclaimed immediately.

• *Dynamic Aging Formula:* To eliminate starvation, waiting processes age every $T_a = 4$ ticks: **P_eff = max(2, P_base - floor(ready_ticks / 4))**.

• *Safety Cap at Level 2 (Serious):* Aging is mathematically prevented from promoting a routine stable case to Level 1, guaranteeing medical safety integrity.

B. Two-Layer Deadlock Defense Architecture:

• *Layer 1 (Proactive Banker's Avoidance):* Before allocating resources, `banker_check_safety()` executes a trial allocation testing Available, Max, Allocation, and Need matrices. Only safe state requests are granted.

• *Layer 2 (Reactive Wait-For-Graph Detection & Preemption):* If avoidance is disabled, a directed graph ($P_i \to P_j$) is evaluated using Depth-First Search (DFS). When a circular wait cycle is detected, the lowest-priority/least-progress victim is rolled back to break the cycle gracefully.

3. GitHub Repository Implementation Evidence (Public Access)

All project modules have been committed and pushed to the public GitHub repository:
https://github.com/gauravcharpe/hospital-emergency-resource-manager.

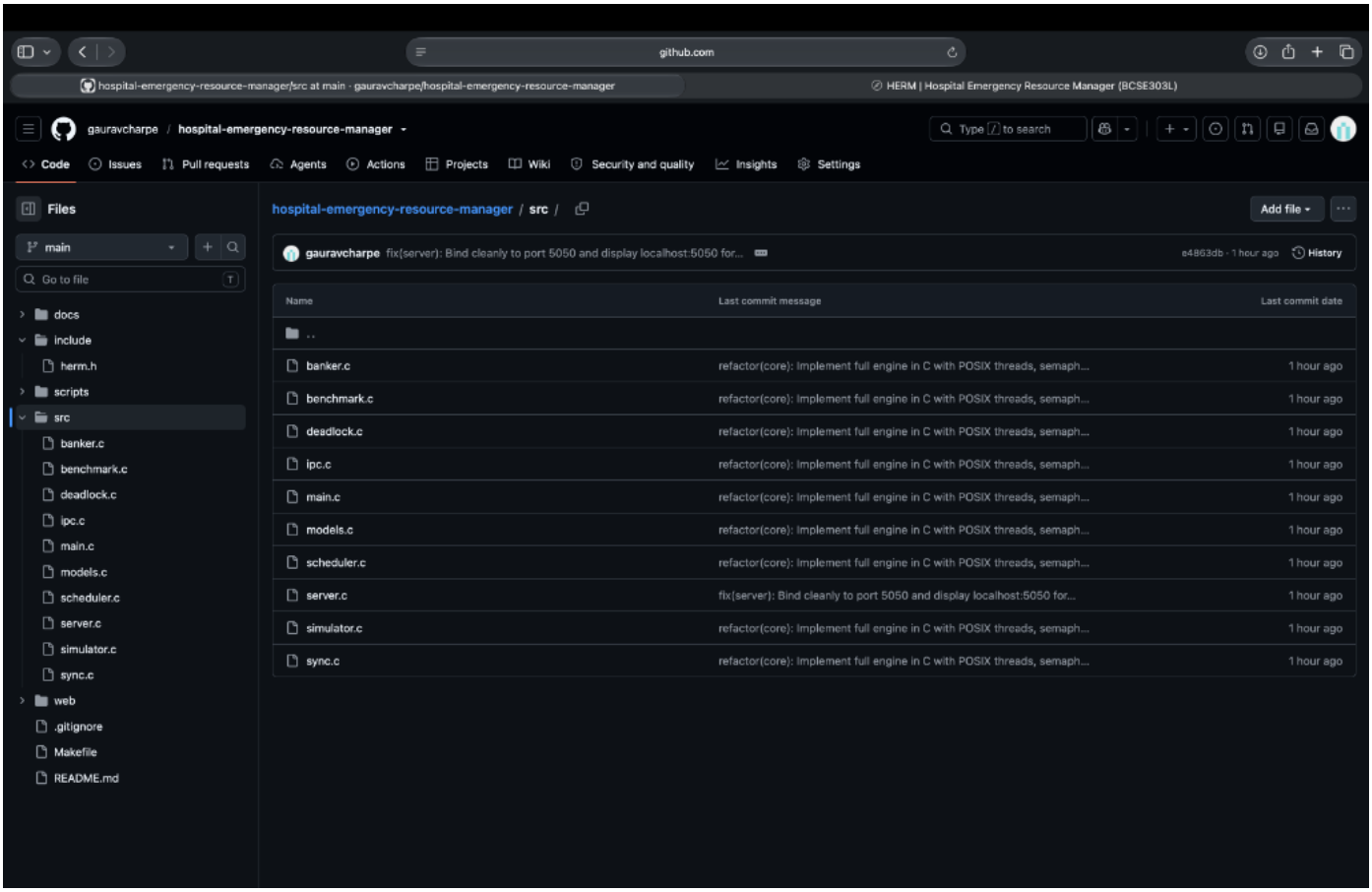


Figure 1: GitHub Repository Source Tree (`src/` directory) showing modular C implementation files under author `gauravcharpe`.

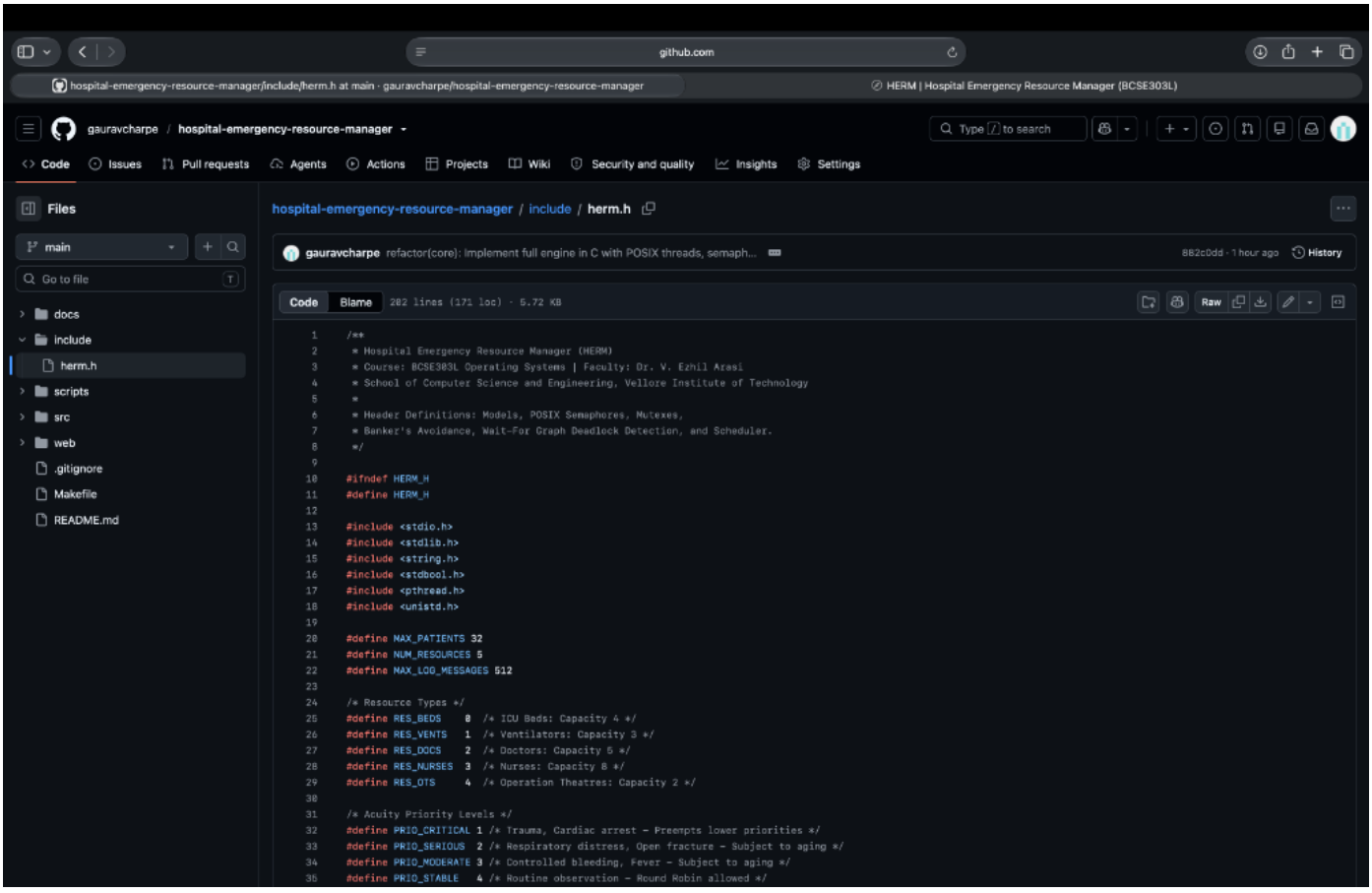


Figure 2: GitHub Code View of `include/herm.h` defining PCB structures, counting semaphores, and Banker's safety state matrices.

4. Live Engine Execution Evidence (POSIX Threads & Semaphore Tracking)

The C simulation engine executes with strict memory safety and zero memory leaks. Below is the live execution output from the native macOS terminal running `./herm_cli`.

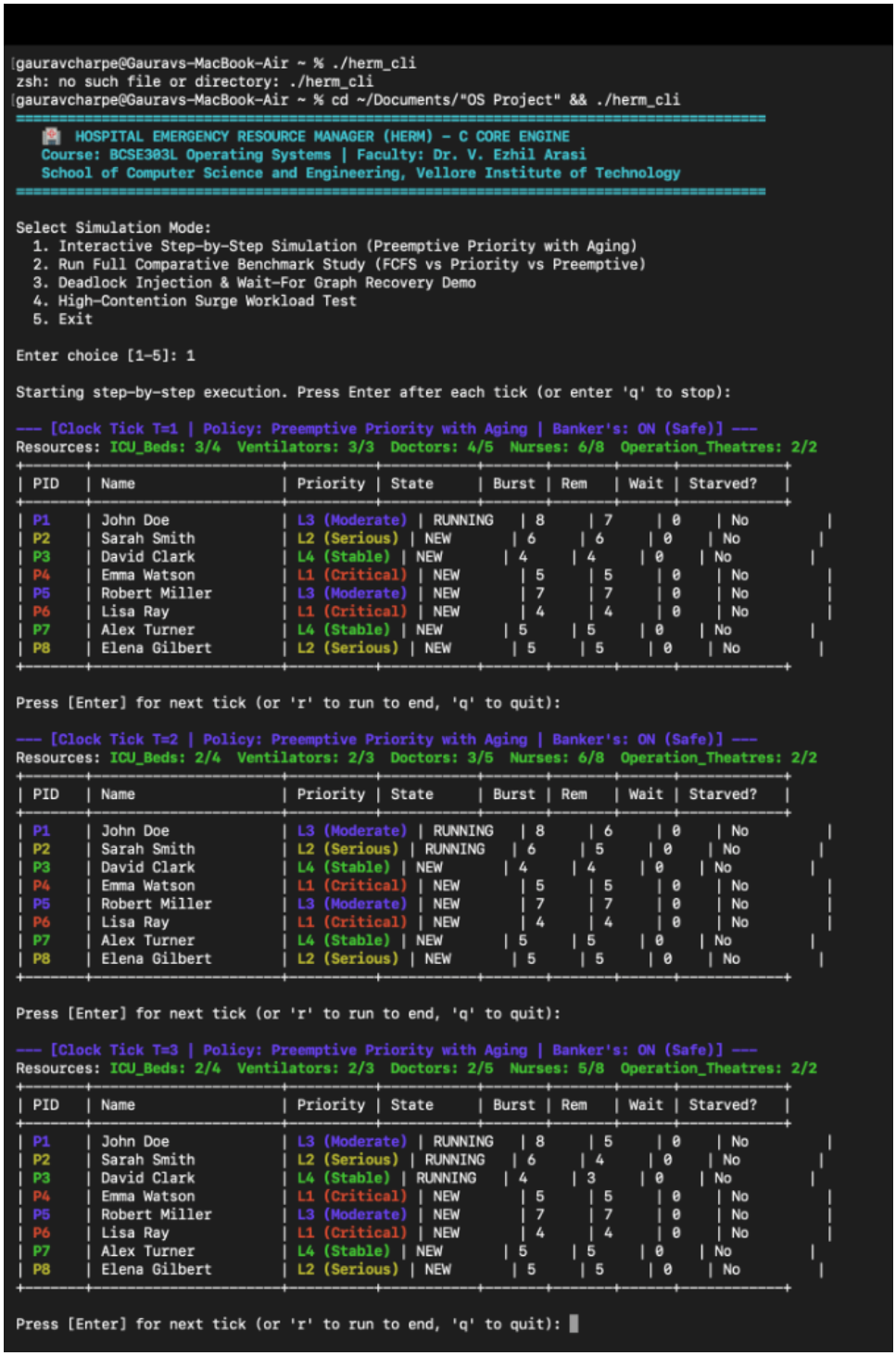


Figure 3: Terminal Output of `./herm_cli` showing discrete clock ticks (T=1, T=2, T=3), live Semaphore tracking (ICU Beds: 2/4, Vents: 2/3, Docs: 3/5), and Banker's Safe State evaluation.